

Part 8: Arrays

1. **How do you declare an array in 8086?** Use `DB` (Define Byte) or `DW` (Define Word) followed by multiple values separated by commas. Example: `nums DB 1, 2, 3, 4`.
2. **How do you access array elements?** Load the array's starting address into a register (like `SI`) and access memory using that pointer. Example: `MOV AL, [SI]`.
3. **Which registers are used for array indexing?** `SI` (Source Index) and `DI` (Destination Index) are the standard registers for moving through arrays, often combined with `BX`.
4. **How do you loop through an array?** Set `CX` to the array size. Inside the loop, access the element, increment the pointer (`SI/DI`), and let the `LOOP` instruction decrement `CX`.
5. **How is `SI` used for array traversal?** `LEA SI, array` loads the start address. `[SI]` accesses the current item. `INC SI` moves to the next byte (or `ADD SI, 2` for words).
6. **How is `DI` used for destination indexing?** It works like `SI` but is typically used as the pointer for where you are writing data to (like copying an array).
7. **How do you sum all elements of an array?** Initialize a sum register (e.g., `AX=0`). Loop through the array adding each element to `AX`: `ADD AL, [SI]`.
8. **How do you find the largest element of an array?** Load the first element as "max". Loop through the rest; use `CMP` to compare current element with max. If greater, update max.
9. **How do you store array elements in memory?** They are stored in contiguous (sequential) memory addresses in the Data Segment.
10. **Can arrays be multidimensional?** Memory is linear (1D). You simulate 2D arrays by calculating the offset: `Address = Base + (Row * RowSize) + Col`.
11. **How do you reverse an array in 8086?** Use two pointers: `SI` at the start, `DI` at the end. Swap the values at `[SI]` and `[DI]`, increment `SI`, decrement `DI`, and repeat until they meet.
12. **How do you copy one array to another?** Set `SI` to source, `DI` to destination, `CX` to size. Loop: Read byte from `[SI]`, write to `[DI]`, increment both.
13. **How do you initialize an array with constants?** Use the `DUP` operator. Example: `arr DB 10 DUP(0)` creates an array of 10 bytes, all initialized to 0.
14. **Which addressing mode is used for arrays?** **Indexed Addressing** (e.g., `[SI]`) or **Based Indexed** (e.g., `[BX+SI]`) are most common.
15. **How do you handle array overflow?** Assembly does not check array bounds automatically. You must manually ensure your index/pointer does not go past the end of the allocated memory.